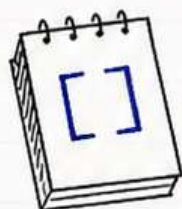
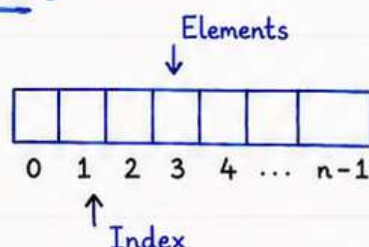


ARRAYS



Master Arrays from Beginner to
FAANG Interview Level



CONTENTS

- ① What is Array? 02
- ② Memory Representation 03
- ③ Time Complexity 04
- ④ Traversal 05
- ⑤ Insertion 06
- ⑥ Deletion 07
- ⑦ Searching 08
- ⑧ Sorting Basics 09
- ⑨ Prefix Sum 10
- ⑩ Sliding Window 11
- ⑪ Two Pointer 12

Think
Visualize
Code

Let's
Crack
Arrays!



★ Why Arrays? ★

Arrays are the foundation of Data Structures and Algorithms.
Master them, and you master problem solving. 😊



Practice. Understand. Apply. Repeat.





ARRAYS



Page 2
of Many

1. WHAT IS AN ARRAY?

★ Definition

An array is a collection of elements of the same data type stored in contiguous memory locations.

Each element is accessed using an index.

★ Real Life Example

Think of an array like a row of lockers in a school.

Each locker has a unique number (index) and stores one item (value).

You can quickly go to any locker using its number.

★ Characteristics

- Stores elements of the same type.
- Elements are stored in contiguous memory locations.
- Accessing any element is fast using its index.
- Indexing usually starts from 0.

★ Advantages

- Fast access to elements ($O(1)$).
- Easy to traverse.
- Cache friendly due to contiguous memory.
- Simple implementation.

★ Disadvantages

- Fixed size (cannot grow/shrink easily).
- Insertion and deletion can be costly.
- Wastage of memory if size is large but elements are few.

★ Diagram

Index	0	1	2	3	4
	10	20	30	40	50

★ Key Points

- ✓ Continuous Memory
- ✓ Fixed Size
- ✓ $O(1)$ Random Access



★ Index

If arr is an array, then

`arr[i]` → accesses the element at index i.

★ Code in C++

```
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {10, 20, 30, 40, 50}; // declaration and initialization
    cout << "Element at index 2: " << arr[2] << endl; // accessing element
    arr[3] = 100; // updating element
    cout << "Updated element at index 3: " << arr[3] << endl;
    return 0;
}
```

Output:

Element at index 2: 30
Updated element at
index 3: 100



ARRAYS



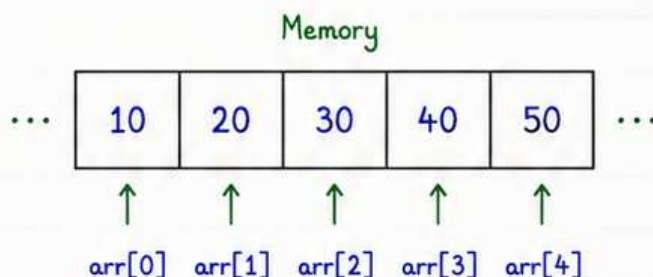
2. MEMORY REPRESENTATION

★ Explain

In an array, all elements are stored in contiguous (continuous) memory locations.

If we know the base address (starting address) and size of each element, we can calculate the address of any element directly.

★ Contiguous Memory



Elements are stored one after another without any gaps.

★ Address Calculation

The address of any element `arr[i]` can be calculated using base address, index (i) and size of each element.

★ Formula

$$\text{Address} = \text{Base} + (\text{Index} \times \text{Size})$$

★ Diagram

Assume: base address = 100

Size of each element = 4 bytes (int)

Address	100	104	108	112	116
Index	0	1	2	3	4
Value	10	20	30	40	50

+4 +4 +4 +4

Each element is stored in the next memory location after the previous one.



★ Why Arrays are Fast?

- ① Because elements are stored in contiguous memory, the address of any element can be calculated directly using a simple formula.
- ② No need to traverse or search for the element.
- ③ This gives $O(1)$ time complexity for access.
- ④ Better cache locality → CPU can fetch nearby elements quickly.



Contiguous Memory + Direct Access = Speed





ARRAYS



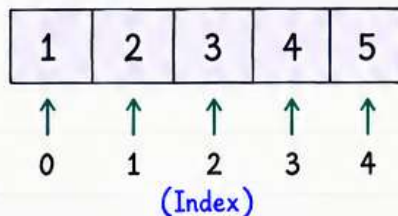
Page 4
of Many

4. TYPES OF ARRAYS

★ 1D Array (One Dimensional)

- Linear collection of elements in a single row.
- Access elements using **one index**.

Diagram:



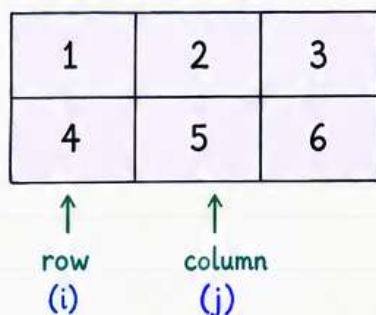
Example (C++):

```
int arr[] = {1, 2, 3, 4, 5};  
for (int i = 0; i < 5; i++) {  
    cout << arr[i] << " ";  
}
```

★ 2D Array (Two Dimensional)

- Collection of elements in a grid.
- Accessed using **two indices** → **row** and **column**.
- Often used for matrices and tables.

Diagram (Matrix):



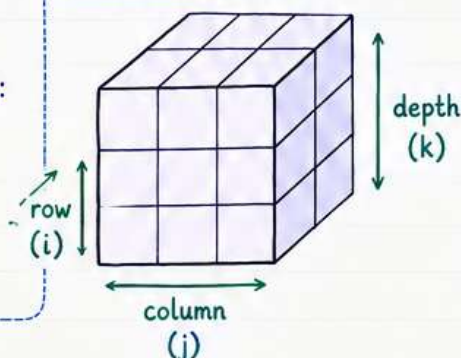
Example (C++):

```
int matrix[2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}  
};  
for (int i = 0; i < 2; i++) {  
    for (int j = 0; j < 3; j++) {  
        cout << matrix[i][j] << " ";  
    }  
}
```

★ 3D Array (Three Dimensional)

- Collection of elements in a cube / box.
- Accessed using **three indices**: **row**, **column** and **depth**.
- Useful for 3D data like images or tensors.

Diagram (Cube):



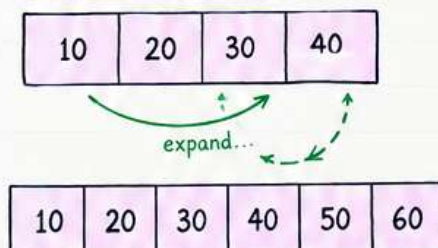
Example (C++):

```
int cube[2][2][2] = {  
    {{1, 2}, {3, 4}},  
    {{5, 6}, {7, 8}}  
};  
for (int i = 0; i < 2; i++)  
    for (int j = 0; j < 2; j++)  
        for (int k = 0; k < 2; k++)  
            cout << cube[i][j][k] << " ";
```

★ 4. Dynamic Array

- Resizable array whose size can change at runtime.
- Implemented using standard containers like **vector** in C++.
- Allocates more memory when needed.

Diagram (Dynamic):



Example (C++):

```
vector<int> arr = {10, 20, 30};  
arr.push_back(40); // size grows  
arr.push_back(50);  
arr.push_back(60);  
for (int x : arr) {  
    cout << x << " ";  
}
```



Key Takeaway:

- 1D → `arr[i]` | 2D → `arr[i][j]` | 3D → `arr[i][j][k]`
- Dynamic Array → grows and shrinks at runtime (vector)

Practice
Builds
Mastery!





ARRAYS



4. TIME COMPLEXITY CHEAT SHEET



Time Complexity tells us how the running time of an operation changes as the size of the array (n) grows.

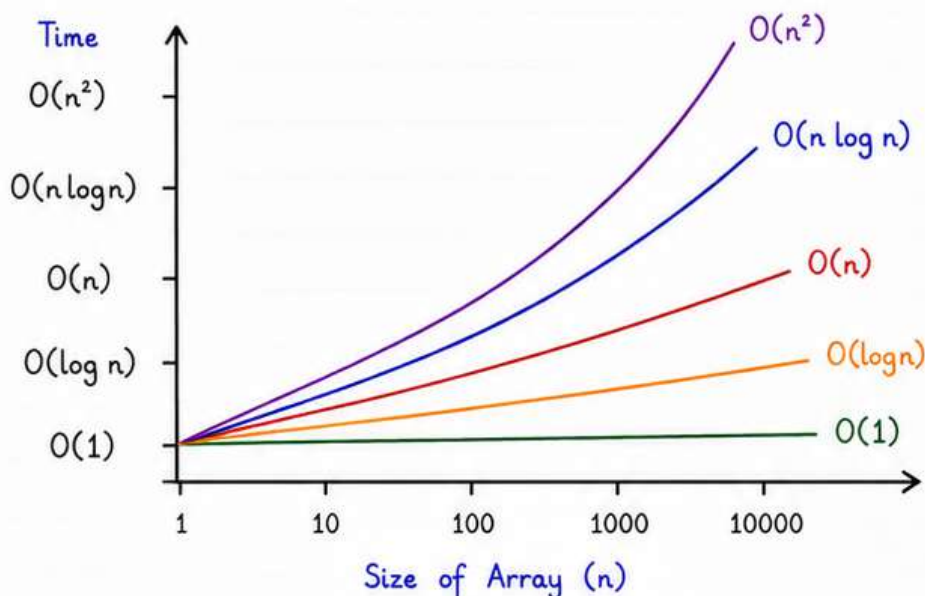
★ Cheat Sheet Table

Operation	Complexity
Access (<code>arr[i]</code>)	$O(1)$
Search (linear)	$O(n)$
Insert at End	$O(1)^*$
Insert at Beginning	$O(n)$
Delete (by value or index)	$O(n)$

Notes:

- $O(1)^*$ means **amortized** $O(1)$. If the array is full and needs resizing, it can take $O(n)$.
- For unsorted arrays, search and delete take $O(n)$.

★ Visual Graph



- $O(n^2)$ Quadratic
- $O(n \log n)$ Log Linear
- $O(n)$ Linear
- $O(\log n)$ Logarithmic
- $O(1)$ Constant

✨ Key Takeaway ✨

Arrays give $O(1)$ access, which makes them very fast for reading elements by index, but inserting or deleting (except at end) can be costly.



Choose the right operation. Use the right data structure.





ARRAYS



7. DELETION IN AN ARRAY



Deletion means removing an element from a specific position. After deletion, remaining elements are shifted to the left to fill the gap.

★ Types of Deletion

- ① Deletion from Beginning (Index 0)
- ② Deletion from Middle (Index k)
- ③ Deletion from End (Index $n-1$)

Shift Operation

After deleting an element, all elements to its right are shifted one position to the left.

Time Complexity : $O(n)$ (worst case)

★ 1. Deletion from Beginning (Index 0)

Before:

0	1	2	3	4
1	2	3	4	5

Delete element
at index 0

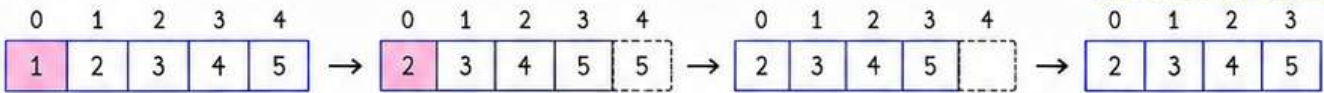
After:

0	1	2	3
2	3	4	5

Steps (Shift Left)

1. Start from index 0
2. $arr[i] = arr[i+1]$
for $i = 0$ to $n-2$
3. Decrease size by 1

Animation:



★ 2. Deletion from Middle (Index k)

Before:

0	1	2	3	4
1	2	3	4	5

Delete element
at index 2

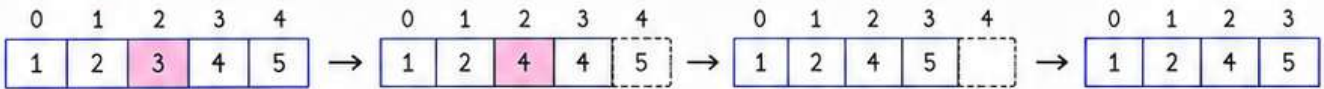
After:

0	1	2	3
1	2	4	5

Steps (Shift Left)

1. Start from index k
2. $arr[i] = arr[i+1]$
for $i = k$ to $n-2$
3. Decrease size by 1

Animation:



★ 3. Deletion from End (Index $n-1$)

Before:

0	1	2	3	4
1	2	3	4	5

Delete element
at index 4

After:

0	1	2	3
1	2	3	4

Steps

1. Remove last element
2. Decrease size by 1
(No shifting required)

Animation:



✨ Key Takeaway ✨

Deletion may require shifting elements to fill the gap.
More shifting → More time.



Delete carefully. Shift smartly. Keep arrays clean and efficient.





ARRAYS



Page 9
of Many

8. SEARCHING IN AN ARRAY



Searching means finding the position (index) of a given element in an array. If it is not found, return -1.

★ 1. Linear Search

In linear search, we check each element one by one from the beginning until the element is found or the array ends.

Example:

arr = [10 , 20 , 30 , 40 , 50]

key = 30

Output: Index = 2

★ 2. Algorithm (Linear Search)

1. Start
2. Read array arr of size n and key
3. For $i = 0$ to $n-1$
 - If $\text{arr}[i] == \text{key}$, return i
4. If key is not found, return -1
5. Stop

★ 3. Dry Run

arr = [10 , 20 , 30 , 40 , 50]

key = 30

Step	i	arr[i]	Comparison	Result
1	0	10	$10 == 30 ?$	No
2	1	20	$20 == 30 ?$	No
3	2	30	$30 == 30 ?$	Yes

Element found at index 2.

★ 4. Example

arr = [5 , 8 , 12 , 7 , 9]

key = 7

Index :	0	1	2	3	4
Value :	5	8	12	7	9



7 is found at index 3.

Output: 3

★ 5. Complexity

- Best Case : $O(1)$
(Element found at first index)
- Worst Case : $O(n)$
(Element found at last index or not found)
- Average Case : $O(n)$
(On average, $n/2$ comparisons)
- Space Complexity: $O(1)$
(No extra space used)



Key Takeaway



Linear Search is **simple** and works on both sorted and unsorted arrays, but it can be **slow** for large arrays.



Search **smartly**. Compare **carefully**. Return the **index**!





ARRAYS



9. BINARY SEARCH (INTRODUCTION)



Binary Search is an efficient searching algorithm for sorted arrays. It repeatedly divides the search interval in half. If the target value equals the middle element, its position is returned. Otherwise, half of the remaining part is eliminated and the search continues on the half in which the target can exist.

★ 1. Requirements

- The array must be **sorted** in either ascending or descending order.
- Allows **random access** to elements (arrays support this).
- Works best on **large datasets** due to its low time complexity.

2. Sorted Array (Ascending Order)

Index :	0	1	2	3	4	5	6
arr :	4	8	15	23	42	56	78

→
Increasing Order

★ 3. How it Works (Diagram)

Target = 23

Step 1:

0	1	2	3	4	5	6
4	8	15	23	42	56	78

↑
↑
↑
L
M
R

mid = 3
arr[mid] = 23
Found!

Step 2:

0	1	2	3	4	5	6
4	8	15	23	42	56	78

↑
↑
↑
L
M
R

If target > mid value,
search in **right half**
(L = 4)

Step 3:

0	1	2	3	4	5	6
4	8	15	23	42	56	78

↑
↑
↑
L
M
R

If target < mid value,
search in **right half**
(R = 4)

★ 4. Mid Calculation

To find the middle index:

$$\text{mid} = \text{low} + \frac{\text{high} - \text{low}}{2}$$

This formula avoids **integer overflow** that can occur in $(\text{low} + \text{high}) / 2$.

★ 5. Example Code in C++

```

int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;    // Not found
}

```

Explanation:

- Initialize **low** and **high** pointers.
- Calculate **mid**.
- Compare **arr[mid]** with **key**.
- If equal, return **mid**.
- If smaller, search in **right half**.
- If greater, search in **left half**.
- If not found, return **-1**.

✨ Key Takeaway ✨

Binary Search is **much faster** than Linear Search.
Use it on **sorted arrays** for best **performance**.



Sort first. Search smartly. Save time!





ARRAYS



10. PREFIX SUM & DIFFERENCE ARRAY



Prefix Sum helps in answering range sum queries quickly.

Difference Array helps in performing range updates efficiently.

★ 1. PREFIX SUM

Concept

Prefix Sum array stores the sum of all elements from index 0 to i .

Formula

Let arr be the array and pre be prefix sum array.

$$pre[0] = arr[0]$$

$$pre[i] = pre[i-1] + arr[i] \quad (\text{for } i \geq 1)$$

Applications

- Range Sum Queries in $O(1)$
- Subarray Sum problems
- Competitive Programming
- Optimization of repeated sum calculations

★ Example

Index (i)	0	1	2	3
$arr[i]$	1	2	3	4
$prefix[i]$	1	3	6	10

Calculation:

$$prefix[0] = 1$$

$$prefix[1] = 1 + 2 = 3$$

$$prefix[2] = 3 + 3 = 6$$

$$prefix[3] = 6 + 4 = 10$$

Range Sum Query using Prefix Sum

Sum of elements from index L to R
 $= prefix[R] - prefix[L-1]$
 (if $L > 0$, else $prefix[R]$)

★ 2. DIFFERENCE ARRAY

Concept

Difference Array is used to perform range updates efficiently.

We update the difference array and then compute the final array using prefix sum.

Range Update

To add value val in range $[L, R]$ (0-indexed):

$$diff[L] += val$$

$$\text{if } (R + 1 < n) \text{ diff}[R + 1] -= val$$

Then build the final array by prefix sum of $diff$.

Applications

- Efficient for many range updates
- Used in problems like:
 - Increment elements in range
 - Add/Assign in intervals
 - Imos Method (Competitive Programming)

Example

Initial Array:

Index	0	1	2	3
arr	1	2	3	4

Perform range update: add 5 to range $[1, 2]$

Step 1: Difference Array ($diff$)

Index	0	1	2	3	4
$diff$	0	0	0	0	0

(size $n+1$)

Step 2: Apply Update

$$diff[1] += 5$$

$$diff[3] -= 5$$

Index	0	1	2	3	4
$diff$	0	5	0	-5	0

Step 3: Prefix Sum of $diff \rightarrow$ Final Array

Index	0	1	2	3
$diff$	0	5	0	-5
$prefix(arr)$	1	6	6	4

Explanation:

- $prefix[0] = 1$
- $prefix[1] = 1 + 5 = 6$
- $prefix[2] = 6 + 0 = 6$
- $prefix[3] = 6 + (-5) = 4$

Final Array after update
(add 5 to $[1, 2]$):

1	6	6	4
0	1	2	3

🌟 Key Takeaway 🌟

Prefix Sum $\rightarrow$ Fast range sum queries.

Difference Array $\rightarrow$ Fast range updates + build final array using prefix sum.





ARRAYS



Page 6
of Many

5. TRAVERSAL OF AN ARRAY



Traversal means visiting each element of the array exactly once in a particular order (usually from first to last).

★ Concept

To process or access every element of an array, we **traverse** it a the first index (0) to the last index (n-1).

Array Example

arr = [10 , 20 , 30 , 40 , 50]

Index : 0 1 2 3 4

Traversal Order : 10 → 20 → 30 → 40 → 50

★ Loop

We use loops to traverse arrays.

The most commonly used loop is the **for loop**.

★ For loop

The for loop **initializes** a counter, checks the **condition**, executes the **block of code**, and **updates** the counter.

Flow of for Loop

Initialization

Condition Check

Execute Block

Update Counter

★ Example code in C++

```
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int n = 5;
    // Traversing the array using for loop
    cout << "Array elements are: " << endl;
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " "; // Access and print each element
    }
    cout << endl;
    return 0;
}
```

Explanation:

- **int i = 0** → start from first index (0)
- **i < n** → run until last index (n-1)
- **i++** → move to next index (1, 2, 3, ..., n-1)
- **arr[i]** → access current element at index **i**

✨ Key Takeaway ✨

Traversal helps us access and process every element of the array.

Use **for** loop to visit each element from index 0 to n-1.



Understand the flow. Write clean loops. Think in indices.





ARRAYS



6. INSERTION IN AN ARRAY



Insertion means adding a new element at a specific position in the array. Since arrays have fixed size, we first shift the elements to the right to make space for the new element.

★ Types of Insertion

- ① Insertion at Beginning
- ② Insertion at Middle
- ③ Insertion at End

Example:

Original Array : 1 2 3 4

Insert X at index 2 (middle)

Result Array : 1 2 X 3 4

★ 1. Insertion at Beginning (Index 0)

Before:

0	1	2	3
1	2	3	4

Insert X
at index 0

After:

0	1	2	3	4
X	1	2	3	4

Steps: 1. Shift all elements one position to the right 2. Insert X at index 0

★ 2. Insertion at Middle (Index k)

Before:

0	1	2	3
1	2	3	4

Insert X
at index 2

After:

0	1	2	3	4
1	2	X	3	4

Steps: 1. Shift elements from index k to n-1 one position to the right
2. Insert X at index k

Key Points:

- Array size must be sufficient to hold the new element.
- Time Complexity: $O(n)$ in worst case (for shifting).
- Insertion at End is **fastest**.
- Insertion in Beginning or Middle requires shifting.

★ 3. Insertion at End (Index n)

Before:

0	1	2	3
1	2	3	4

Insert X
at index 4

After:

0	1	2	3	4
1	2	3	4	X

Steps: 1. Insert X at index n (next position)
2. No shifting required

✨ Key Takeaway ✨

Insertion may require shifting elements to create space.
More shifting → More time.

